Studio Nx

Backend(FastAPI/Python) 36 — to_me_response()

2026-10-07

Backend(FastAPI/Python) 36 — `to_me_response()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보 — 내 정보 응답 만들기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	35 `profile_image_url()` · 37 `update_me()`

1. 하는 일

DB 의 회원 한 줄(`User`)을 화면에 보낼 **내 정보 응답(`MeResponse`)** 으로 바꾼다. 정해 둔 7칸만 담는다.

```
{
  "user_id": 85,
  "email": "demo@example.com",
  "nickname": "demo_user",
  "role": "USER",
  "profile_image_url": "/api/v1/profile-images/3f9c….jpg",
  "created_at": "2026-10-07T01:12:30",
  "last_login_at": "2026-10-07T09:30:00"
}
```

담지 않는 칸 (User 에는 있음)	이유
`password_hash`	해시라도 밖으로 나가면 안 됨 — 무작위 대입으로 원래 비밀번호를 찾는 데 쓰일 수 있음
`status` · `deleted_at`	이 응답은 로그인한 **활동 회원**만 받음 (06 이 이미 막음)
`privacy_agreed_at` · `privacy_version` · `storage_*` 등	회원 정보 화면이 쓰지 않음. 용량은 P1-16 대시보드 응답이 따로 줌

2. 언제 쓰나

쓰는 곳 (P1-08 라우터)	
`GET /users/me` (41)	내 정보
`PATCH /users/me` (41)	닉네임을 바꾼 뒤 바뀐 정보
`POST · DELETE /users/me/profile-image` (42)	사진을 바꾸거나 지운 뒤 바뀐 정보 (새 `profile_image_url`)

화면: `userinfo.js` `loadMe`(P1-10), `mypage.js`.

`user_id` 를 넣는 것은 괜찮다 — 이 응답은 **토큰을 가진 본인**에게만 간다. 탈퇴 추적 방지 ③ 은 "남의 번호로 조회하는 공개 API" 를 막는 것이다.

3. 코드

import (파일 맨 위 — 35 와 같음, 추가 없음)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
```

이 함수는 **`User`** · **`MeResponse`** 와 35 `profile_image_url` 을 쓴다.

함수

```
def to_me_response(user: User) -> MeResponse:
    return MeResponse(
        user_id=user.user_id,
        email=user.email,
        nickname=user.nickname,
        role=user.role,
        profile_image_url=profile_image_url(user),
        created_at=user.created_at,
        last_login_at=user.last_login_at,
    )
```

문법 풀이

import 문법	뜻
`from app.schemas.user import MeResponse`	템플릿에 있던 응답 모양 (pydantic `BaseModel`)

식	뜻
`MeResponse(user_id=…, …)`	이름 붙인 인자(키워드 인자)로 칸마다 값을 넣어 만듦. 빠진 칸 · 틀린 형식이면 pydantic 이 바로 오류
`role=user.role`	DB 는 `UserRole.USER`(StrEnum). `StrEnum` 은 글자이기도 해서 응답 칸 `str` 에 그대로 들어감 → JSON 에 `"USER"`
`created_at` (`datetime`)	FastAPI 가 JSON 으로 바꿀 때 `"2026-10-07T01:12:30"` 모양 글자로

4. 흐름

1. 회원의 칸 6개를 그대로 옮긴다.
2. 사진은 35 로 주소를 만든다.
3. 응답 객체를 돌려준다.

5. 왜 이렇게

선택	이유
칸을 **하나씩** 적음	`MeResponse.model_validate(user)`(ORM 에서 자동으로 꺼내기)도 되지만, 그러려면 스키마에 `from_attributes` 설정이 필요하고 `profile_image_url` 은 DB 칸이 아니라 따로 계산해야 한다. 적어 두면 무엇이 나가는지 한눈에 보인다
응답 칸을 스키마로 고정	나중에 `User` 에 칸이 늘어도 응답에 저절로 섞이지 않는다 — 시험이 "칸은 7개뿐" 을 확인한다

선택	이유
DB 를 다시 읽지 않음	부르는 쪽이 이미 최신 `User` 를 가지고 있다 (수정 뒤에는 37 · 38 이 `refresh`)

6. 시험 (`tests/test_user_service.py` 에 3개 추가 · 모두 8개, 실제 MySQL)

시험	확인한 것
새 회원	7칸 값이 DB 와 같음, `role` `"USER"`, 사진 · 마지막 로그인 `None`, `created_at` 은 DB 가 채운 값
응답 칸	정확히 7개. JSON 안에 비밀번호 해시 글자 · `password` 가 없음
관리자 · 사진 · 로그인 기록	`role` `"ADMIN"`, `last_login_at` 그대로, 사진 주소 `/api/v1/profile-images/{32자}.png`

전체 시험 242개 통과, `ruff` 검사 통과. 시험에서 만든 회원은 시험이 끝나면 되돌려진다 (conftest 의 savepoint).